ArcFS: A Content-Addressable FUSE Filesystem with Inline Deduplication, Copy-on-Write Versioning, and Semantic Tag Navigation

Anandkumar N S, Dhakkshin S R, Kishoreadhith V, M Raj Ragavender, Rithvik K *Department of Computer Science and Engineering*
PSG College of Technology, Anna University
Coimbatore, India – 641 004
{22z209, 22z215, 22z232, 22z233, 22z253}@psgtech.ac.in Dr. Arul Anand N (Faculty Guide) *Department of Computer Science and Engineering*
PSG College of Technology
Coimbatore, India – 641 004

***Abstract*—Traditional filesystems store data by location, which prevents native deduplication, makes versioning expensive, and confines every file to a single position in the directory hierarchy. We present ArcFS, a user-space filesystem implemented in Rust that integrates three storage optimizations into a single transparent write path: (i) content-defined chunking (CDC) via a FastCDC Gear-hash engine with a 256-entry lookup table, producing chunks between 2 KB and 64 KB at a 4 KB probabilistic target; (ii) SHA-256 content-addressable storage (CAS) with per-chunk Zstandard level-3 compression, where the hash is computed over uncompressed bytes before any compression transform; and (iii) copy-on-write (CoW) snapshots implemented at the recipe-pointer level, making snapshot creation O(1) in chunk bytes (no data is copied) at an O(n_i) sled cost proportional to live inode count n_i.**

A semantic virtual filesystem layer (TagFS) overlays the namespace and resolves composite tag paths through lazy set-intersection over sled-backed tag indexes, with in-memory virtual-inode caches for tag directories and files. This allows files to appear under multiple tag hierarchies without data duplication. The write-back page cache is implemented as a `HashMap<u64,(Vec<u8>,bool)>` bounded at 1,024 entries by an LRU policy maintained through a companion `VecDeque<u64>`, deferring the computationally expensive CAS pipeline to eviction, `fsync`, or file release.

ArcFS is evaluated against ext4, Btrfs, and a bindfs FUSE pass-through baseline across three benchmark classes (responsive, durable, integrity) and a four-profile disk-usage suite. In the responsive (asynchronous) class, ArcFS achieves 4,275 MB/s sequential write throughput, a 9.2× advantage over ext4, by coalescing writes entirely in the bounded-memory cache. On storage efficiency, it reduces identical-file storage by 83.7%, cuts mixed-workload disk consumption by 57.6% relative to ext4, and holds snapshot storage costs within 1.3× of kernel-native Btrfs CoW. All CRC32c integrity verifications return zero errors, confirming lossless reconstruction through the full chunking, compression, and decompression pipeline. We additionally characterize the system's design trade-offs and limitations, including the offline nature of garbage collection, the O(n) LRU-update cost, and the residual metadata growth from snapshot recipe entries. ArcFS serves as a concrete foundation for cloud storage primitives—container image layer caches, build artifact stores, and scientific data archives—where inline deduplication and semantic navigation reduce both storage cost and retrieval latency across large, redundant corpora.

***Index Terms*—content-addressable storage, data deduplication, content-defined chunking, FUSE, copy-on-write, semantic filesystem, garbage collection, Rust**

I. Introduction

Modern storage workloads simultaneously demand storage efficiency, revision history, and flexible retrieval. Conventional Linux filesystems address each requirement in isolation. Ext4 overwrites data in place, supports no deduplication, and delegates versioning entirely to application-layer tools. Btrfs adds copy-on-write snapshots and optional transparent compression but lacks cross-file content deduplication without an offline pass through a tool such as `duperemove`. Neither system supports semantic organization beyond the directory tree.

Application-level deduplicators such as Restic [1] and BorgBackup [2] perform content-defined chunking effectively: both tools split archive content into variable-size chunks that are hashed and stored in flat content-addressed repositories, achieving significant space savings for backup workloads. Their benefits, however, do not extend to the filesystem layer: the underlying storage still writes every byte verbatim, and space savings require all applications to opt into the specific backup agent. Cloud storage systems face the same gap at scale—tenant VMs writing to shared block storage, build pipelines writing duplicate artifacts, and scientific workflows producing near-identical output files all generate redundant data that per-application tools cannot collectively deduplicate. Building these optimizations into the filesystem layer removes the coordination burden from individual applications and makes space savings automatic and transparent across all writers, regardless of language, framework, or access pattern.

A Filesystem in Userspace (FUSE) implementation offers a practical path to experimenting with such a design without kernel modifications, but three engineering challenges arise. First, every VFS call crosses the kernel/user boundary twice, imposing context-switch latency documented at up to 83 round trips per file read [3]. Second, deduplication systems that share

chunk payloads across inodes require careful metadata management to maintain consistency across concurrent writers [4]. Third, in-memory hash tables mapping chunk digests to their CAS addresses grow unboundedly with dataset size unless the design imposes an explicit eviction policy [5].

ArcFS addresses all three challenges within a single Rust prototype. A write-back page cache—typed as `HashMap<u64, (Vec<u8>, bool)>` and bounded at 1,024 entries by an LRU eviction policy maintained through a companion `VecDeque<u64>`—coalesces small writes before committing them to the CAS pipeline, reducing the per-write FUSE round-trip count. The sled embedded database [6] provides ACID key-value semantics with a prefixed key schema that encodes inode-to-recipe mappings as separate relations from chunk payloads, isolating logical metadata from immutable physical data. The LRU bound caps the number of cached inodes; the total byte footprint still depends on the sizes of buffered files.

The contributions of this paper are as follows:

1) **Unified pipeline design.** A five-layer storage pipeline—FUSE interface, write-back LRU cache, FastCDC chunking engine, SHA-256 CAS storage, and sled metadata engine—in which each layer is independently testable and adjacent layers communicate through narrow byte-stream or hash-set contracts. We show that these layers can be composed in a single user-space daemon without kernel modifications.
2) **Per-file chunker isolation as a design choice.** The decision to instantiate a fresh `Chunker` per file ingestion (rather than sharing rolling hash state across files) is a deliberate design choice that prevents cross-file boundary interference at the cost of forgoing any cross-file boundary correlation that might improve deduplication on structurally similar files (e.g., log rotation). We characterize the tradeoff and show that the resulting deduplication ratio of 83.7% on identical-file workloads remains competitive with kernel-level reflink.
3) **Mark-and-sweep GC over a dual-store system.** The two-phase offline garbage collector—sled prefix scan for the active hash set, followed by a CAS directory walk—is a design choice that avoids per-chunk reference counting (which would require atomic increments on every dedup hit) in favor of a periodic batch sweep. We characterize the memory cost of the active set ($64\,\mathrm{B}\times$ unique referenced chunks) and identify the snapshot-deletion recipe-leak that makes the GC conservative in practice.
4) **TagFS virtual ontology engine.** A semantic navigation layer that resolves composite tag paths via lazy set-intersection over sled-backed tag indexes, with virtual inode caches populated lazily at `lookup`/`readdir` time (no mount-time materialization). The control interface accepts live tag mutations via a sentinel file without requiring a remount cycle.
5) **Benchmark methodology separating acceptance-path from persistence-path cost.** A three-class benchmark campaign (responsive, durable, integrity) plus a four-profile disk-usage suite, showing that the $9.2\times$ write throughput advantage over ext4 is attributable entirely to write-back coalescing and collapses to near-parity under durable fsync semantics.
6) **Honest limitation characterization.** A quantified analysis of the design's failure modes: the GC concurrent-use safety window, the O(n) LRU-update cost at 1,024-inode cache capacity, the permanent storage growth from snapshot recipe leaks, the absence of on-read hash verification, and the single-threaded FUSE dispatch model.

II. Background and Motivation

A. Content-Defined Chunking

Variable-size content-defined chunking partitions a byte stream into chunks whose boundaries are determined by the content itself rather than fixed offsets. The central virtue of CDC is boundary stability: inserting or deleting k bytes at position p perturbs at most the two chunks straddling p; all other boundaries remain unchanged, preserving their SHA-256 hashes and therefore their CAS blobs. Fixed-size chunking lacks this property entirely: a single-byte insertion at offset 0 shifts every downstream boundary by one byte, destroying all hash matches for a file that is otherwise 99.9% identical.

The dominant boundary-detection approach uses a rolling hash. The classic Rabin fingerprint polynomial hash used by LBFS [7] processes one byte per polynomial evaluation. FastCDC [8] replaces polynomial arithmetic with a single 256-entry lookup table (the Gear matrix), reducing boundary evaluation to one table lookup and one left-shift-and-add per byte while preserving boundary entropy.

B. Content-Addressable Storage

A content-addressable store (CAS) maps content to a canonical identifier derived entirely from the content, typically a cryptographic hash. Two byte strings with the same hash map to the same identifier; therefore, storing the second copy requires only a pointer, not additional bytes. Deduplication is implicit: the store checks for the existence of the destination path before writing. Venti [9], the Plan 9 network storage server, pioneered the use of SHA-1 CAS for archival storage, demonstrating that a flat hash-addressed store is a natural substrate for deduplication and that immutable blobs enable safe sharing across divergent revisions.

C. Write-Back Caching in User-Space Filesystems

FUSE imposes a per-request kernel/user round trip. For a workload that issues n write system calls, a naive FUSE implementation triggers n pipeline invocations, each crossing the boundary twice. Write-back caching coalesces these: each write updates an in-memory buffer and returns immediately to the caller; the pipeline runs only at eviction, `fsync`, or file close. The throughput ceiling shifts from physical-media latency to FUSE IPC bandwidth, which is bounded by the kernel ring buffer and CPU, not the disk.

III. Related Work

A. Semantic and Tag-Based Filesystems

Gifford et al. [10] proposed semantic file systems in which handler programs automatically derive file attributes from content, decoupling logical organization from physical location. Bloehdorn and Völkel [11] demonstrated tag-based hierarchical navigation through explicit metadata annotations at the VFS level, showing that virtual directory construction from tag intersections is feasible without modifying application binaries.

ArcFS extends this approach in two ways. First, intersections are computed lazily at `readdir`/`lookup` time by querying sled-backed tag indexes and caching virtual inode identities in memory, rather than materializing all tag intersections on disk. Second, the tag control file (`.tagfs_ctl`) at the filesystem root accepts line-oriented `set` and `del` commands without requiring an unmount cycle, enabling live tag mutation from any POSIX-compatible process.

B. Content-Defined Chunking and Deduplication

The Low-Bandwidth Network Filesystem (LBFS) [7] introduced content-defined chunking with Rabin fingerprints to suppress redundant network transfers. Zhu et al. [5] demonstrated that combining CDC with content-addressed storage substantially reduces backup storage; their Data Domain system processes chunk fingerprints through an in-memory bloom filter and a similarity index before issuing disk I/O, bounding memory even for petabyte-scale datasets.

BorgBackup [2] applies CDC deduplication at the application layer, chunking archive content into variable-size segments that are hashed, encrypted, and stored in a flat repository. Unlike ArcFS, Borg operates outside the filesystem and requires all clients to use the same backup agent; space savings are not shared across arbitrary writers.

ArcFS adopts the FastCDC algorithm [8] with a 256-entry Gear-hash lookup table, which replaces the polynomial arithmetic of Rabin fingerprinting with a single table lookup per byte, lowering chunking CPU cost while preserving boundary entropy. The Gear matrix is built at compile time via a `const fn` that chains `splitmix64`-derived 64-bit values seeded from a fixed 64-bit constant.

C. Copy-on-Write Filesystems

Rosenblum and Ousterhout [12] established the log-structured filesystem, providing the structural foundation for efficient CoW semantics. Btrfs and ZFS operationalize this idea for production workloads, sharing unchanged extents across snapshot subvolumes through kernel-level reference-counted B-tree nodes.

ArcFS applies the same principle at the recipe level: a snapshot is a `SnapshotMetadata` record (name, Unix timestamp, root inode ID) persisted in sled. The snapshot's inode tree is materialized by deep-cloning live inode metadata—assigning new inode IDs while copying recipe pointers—so snapshot and live namespaces share CAS blobs without copying them. Snapshot creation is O(1) in CAS operations (no chunk bytes are duplicated); the sled-level cost is O(n_i) where n_i is the number of live inodes, as each inode's `ino_meta:` and `ino_recipe:` records must be cloned. For a 200 MB base with ∼1,000 inodes, this represents approximately 2,000 sled writes plus flush calls, which completes in milliseconds on modern NVMe storage.

D. Write-Optimized Data Structures and Storage Engines

BetrFS [13] demonstrates that B$^\varepsilon$-trees, a write-optimized variant of B-trees that buffers updates in internal nodes and flushes them in large batches, can substantially improve sequential and random write throughput for filesystem metadata compared to conventional B-tree layouts. ArcFS uses sled, a concurrent B-tree engine written in Rust [6], rather than a write-optimized tree, making metadata write performance proportional to the sled B-tree's update cost plus the cost of `flush()`. Adopting a write-optimized metadata engine is a natural direction for future work.

WiscKey [14] separates B-tree index keys from their associated values, storing values in a circular log and indexing them by key in a compact LSM tree. This design substantially reduces write amplification for large values relative to log-structured merge trees that co-locate keys and values. ArcFS implicitly separates chunk data (the CAS blobs on the POSIX filesystem) from chunk identifiers (SHA-256 hex strings in sled), achieving a similar key-value separation at the storage-engine boundary. Unlike WiscKey, CAS blobs in ArcFS are immutable once written; there is no garbage-collection pressure from value overwriting.

E. Distributed and Networked Filesystems

Ceph [15] provides a distributed filesystem built on RADOS, a reliable, autonomous distributed object store. CephFS exposes a POSIX-compatible interface over RADOS objects, scaling metadata across multiple MDS daemons using a dynamic subtree partitioning algorithm. ArcFS occupies the opposite end of the deployment spectrum: a single-node user-space daemon with no distributed coordination layer. The CAS storage directory in ArcFS could in principle be replaced by a distributed object store (e.g., a RADOS pool or an S3-compatible endpoint), extending ArcFS into a lightweight distributed deduplication gateway; this is reserved for future work.

F. User-Space Filesystem Performance

Vangoor et al. [3] systematically characterized FUSE overhead and showed that request coalescing can recover a large fraction of the penalty. Miller et al. [16] demonstrated Rust-based in-kernel modules that eliminate context-switch cost entirely, approaching native throughput. ArcFS operates in user space and uses a write-back cache as its primary mitigation strategy, trading some durability-path latency for substantial acceptance-path throughput.

G. Applicability to Cloud Storage Workloads

The design patterns in ArcFS address workloads that arise repeatedly in cloud infrastructure. Container image layers are

highly redundant (base images shared across thousands of containers) and benefit directly from CDC deduplication: each layer's files are chunked on write, and shared base-layer content produces SHA-256 hash collisions that the CAS existence check silently deduplicates. Build artifact caches (e.g., compiled binaries and test results across CI pipelines) exhibit near-identical content with small diffs; CDC boundary stability means that a single-line source change affects at most two chunks, leaving the rest as CAS hits. Scientific data archives (genomics, climate simulation) store large files with high inter-file similarity; the 4 KB target chunk size matches the typical page size and produces efficient deduplication across related datasets. The TagFS virtual navigation layer provides a semantic retrieval interface well-suited to multi-dimensional scientific metadata (e.g., `@tags/species/homo-sapiens/2026/`) that does not map cleanly onto a single directory hierarchy. These use cases motivate the distributed CAS and metadata backends discussed in Section VII.

IV. System Architecture

ArcFS couples seven cooperating subsystems: the FUSE handler, the write-back LRU cache, the FastCDC chunking engine, the SHA-256 CAS storage layer, the sled metadata engine, the CoW versioning engine, and the TagFS virtual ontology. The subsections below describe each layer in implementation-verified detail; benchmark evidence appears in Section V.

A. FUSE Handler and Inode Registry

The system implements the `fuser::Filesystem` trait, routing POSIX VFS calls to two data structures: a per-process inode registry typed as `Arc<RwLock<HashMap<u64, Arc<RwLock<Inode>>>>>` and a persistent sled database.

Inode identifiers use two ranges by convention. Virtual TagFS inodes start at `VIRTUAL_INODE_START` = 1,000,000 and are allocated from an `AtomicU64` counter initialized at mount time; they exist only in the running daemon's memory and are never persisted. Real inodes are allocated from a separate `next_inode` counter (initialized to 100) and are expected to remain below this boundary in current deployments, though the prototype does not enforce a hard separation. Five reserved identifiers anchor the fixed namespace: root (1), `.snapshots/` (2), snapshot-create sentinel (3), `@tags/` (4), and the TagFS control file (5).

File creation executes a five-step sequence: (i) monotonic ID assignment via `AtomicU64::fetch_add`, (ii) `InodeMetadata` instantiation, (iii) dual database write: one `sled::Db::insert` for the metadata record under key `ino_meta:<id>` and one for the directory-entry edge under `dirent:<parent>:<name>`, (iv) empty `FileRecipe` pre-allocation, and (v) registry injection. Each persistence call ends with `Tree::flush()`, which calls `sync_all()` on the underlying file; steps are durable individually but are not atomic as a group. A daemon crash between steps leaves state that is partially written to sled; the registry is rebuilt by scanning `dirent:` edges and loading the corresponding `ino_meta:` records at the next mount. Orphaned inode metadata can therefore remain in the database until an explicit metadata sweep is implemented (the current GC only reclaims CAS blobs).

Lock ordering is strict and enforced by code review to prevent deadlocks under concurrent FUSE workloads. The mandatory acquisition order is:

1) Global registry `RwLock` (shortest critical section; dropped before deeper locks are acquired).
2) Per-inode `RwLock` (acquired only after the registry lock is released).
3) Page-cache `RwLock` (acquired only in isolation, never while a per-inode lock is held).

Snapshot creation and tag mutations both acquire locks in this order. In the write path (`write()` handler), the page-cache write lock is released before the registry and per-inode locks are acquired for attribute update, ensuring sequential rather than nested lock acquisition.

At boot, `hydrate_tree()` performs a depth-first reconstruction of the live inode tree by iterating over `dirent:` prefix keys in sled for each directory, loading the corresponding `ino_meta:` record, reconstructing the in-memory `Inode` object with the correct `FileType` and `size` fields, and inserting it into the registry. The `next_inode` counter is advanced past the highest observed inode ID to prevent ID reuse.

B. Write-Back LRU Cache

The cache is the primary mechanism for mitigating FUSE context-switch overhead. It is implemented as two coordinated structures.

a) Data map.: An `Arc<RwLock<HashMap<u64, (Vec<u8>, bool)>>>` keyed by inode ID. The `u64` key is the inode identifier; the `Vec<u8>` value holds the accumulated write buffer for that inode; the `bool` flag records whether the entry is dirty (i.e., the buffer contains data not yet committed to the CAS layer and sled).

b) LRU order.: An `Arc<RwLock< VecDeque<u64>>>` holding inode IDs in least-recently-used to most-recently-used order (front to back). This structure tracks access order so that eviction always removes the inode whose last write occurred furthest in the past.

The cache capacity is 1,024 entries (constant `PAGE_CACHE_CAPACITY`).

c) Write path.: A FUSE `write` call locates the target inode's entry using `HashMap::entry().or_insert_with()`, which performs a single-probe O(1) lookup. If the inode is not yet cached, the file's current content is loaded from sled and the CAS store via `read_file_by_id()`; this avoids data loss from partial overwrites. The incoming bytes are then overlaid onto the buffer with `copy_from_slice()`, the dirty flag is set to `true`, and control returns to the kernel without any disk I/O.

d) LRU maintenance.: Every cache access invokes `touch_cache_entry(ino)`, which acquires the `VecDeque` write lock, scans the deque linearly for the target inode ID, removes it from its current position

(O(n) in the worst case, where n is the number of cached inodes), and pushes it to the back (MRU position). This O(n) scan is the dominant cost of the LRU mechanism under high-concurrency workloads with many cached inodes, though in practice the deque is bounded at 1,024 entries, capping the scan at 1,024 comparisons.

e) Eviction.: `evict_under_pressure()` runs after each write and loops until the cache size is at or below capacity:

1) Pop the front of the `VecDeque` to obtain the least-recently-used inode ID.
2) Look up that ID in the `HashMap`.
3) If the entry is dirty (flag = `true`), call `flush_inode_cache(ino)`: read the buffer from the `HashMap`, push it through the full CAS pipeline (FastCDC → SHA-256 → zstd → disk), commit the updated recipe to sled with `flush()`, and mark the entry clean.
4) Remove the entry from the `HashMap` unconditionally.

Clean entries are silently discarded; only dirty entries trigger CAS pipeline invocations.

f) Flush triggers.: The pipeline activates at three points in addition to capacity eviction: (i) the application calls `fsync(2)`, routing through the `fsync()` FUSE handler, which calls `flush_inode_cache` for the target inode; (ii) the kernel calls `release` on the last file-descriptor close, routing through the `release()` handler; and (iii) `flush_all_dirty_cache()` is called before snapshot creation to ensure all dirty buffers are committed before the snapshot inode tree is cloned.

C. FastCDC Chunking Engine

a) Gear-hash table construction.: The 256-entry `GEAR_MATRIX` is a compile-time constant built by `build_gear_matrix()`, a `const fn` that chains `splitmix64`-derived 64-bit values. Starting from a fixed 64-bit seed `0x1234_5678_9ABC_DEF0`, each entry is computed as:

$$GEAR[i] \leftarrow \texttt{splitmix64}(seed_{i-1} \oplus (i \times \phi_{64})) \quad (1)$$

where $\phi_{64} = \texttt{0x9E3779B97F4A7C15}$ is the 64-bit Fibonacci hashing constant (the golden ratio scaled to fill 64 bits). The XOR with the index-scaled golden ratio ensures that entries are uniformly distributed even for small indices, and the `splitmix64` bijection preserves that distribution. The table is statically embedded in the binary; no runtime initialization is needed.

b) Rolling hash update and boundary detection.: The rolling hash state h (initialized to 0) updates for each byte b_i as:

$$h \leftarrow (h \ll 1) + GEAR[b_i] \quad (2)$$

where the shift is a left shift by one bit. A chunk boundary is declared when $(h \ \& \ MASK) = 0$, where $MASK = TARGET\_CHUNK\_SIZE - 1 = 4{,}095$ (i.e., the lower 12 bits of h are all zero). The boundary probability is $1/4096$, giving an average chunk size of 4 KB for uniformly distributed data. A hard minimum of 2 KB prevents degenerate boundaries: the `should_cut()` function returns `false` unconditionally until `current_chunk_size >= MIN_CHUNK_SIZE`. A hard maximum of 64 KB ensures progress regardless of content: the function returns `true` unconditionally when `current_chunk_size >= MAX_CHUNK_SIZE`.

c) Intra-file boundary reset.: After each chunk boundary, the chunker's state is reset to zero via `reset()`, which sets `self.hash = 0`. This ensures that the rolling hash for the next chunk within the same file begins from a clean state, so the position of one boundary does not bias the positions of subsequent boundaries beyond the extent of the rolling window.

d) Cross-file isolation.: ArcFS guarantees that the rolling hash state of one file never influences the chunk boundaries of a subsequent file. This guarantee is structural: `create_recipe_from_data()` calls `Chunker::new()` once at the start of each ingestion, allocating a fresh `Chunker` with `self.hash = 0`. When the ingestion completes, the chunker is dropped; no state is retained between calls. The `reset()` call between intra-file chunks is therefore complementary but distinct: it zeroes the hash within a file's processing loop, while the per-call instantiation prevents any cross-file contamination. Algorithm 1 summarizes the complete boundary-detection procedure.

Algorithm 1 CDC Boundary Detection (`create_recipe_from_data()`)

Require: byte stream B; MIN = 2048, MAX = 65536, MASK = 4095
1: $h \leftarrow 0$; $buf \leftarrow []$; $recipe \leftarrow []$
2: **for all** byte $b \in B$ **do**
3: $h \leftarrow (h \ll 1) + GEAR[b]$; buf.push(b)
4: **let** $cut \leftarrow |buf| \geq$ MAX **or** ($|buf| \geq$ MIN **and** $h \& MASK = 0$)
5: **if** cut **then**
6: $hash \leftarrow \texttt{sha256}(buf)$; `cas.write`($buf$)
7: $recipe$.push($hash$); $buf \leftarrow []$; $h \leftarrow 0$
8: **end if**
9: **end for**
10: **if** $|buf| > 0$ **then**
11: flush tail chunk (same steps as lines 5–7)
12: **end if**
13: **return** $recipe$

D. Content-Addressable Storage

After chunking, each chunk is processed by `write_chunk()`. The SHA-256 digest is computed over the *uncompressed* bytes before any compression transform:

```
let hash_string = hex::encode(Sha256::digest
    (data));
```

This preserves the invariant that content identity is independent of the compression codec or its version: upgrading from zstd level 3 to level 6 would change the compressed bytes but leave all SHA-256 hashes unchanged.

The chunk is stored at path `cas/{hash[0:2]}/{hash[2:]}` under the storage root directory. The two-level sharding—using the first two hexadecimal characters as a subdirectory name—limits each directory to at most 256 subdirectories and 16^{62} files

TABLE I
SLED DATABASE KEY NAMESPACE

Key Pattern	Value
`ino_meta:<id>`	`InodeMetadata` (bincode)
`ino_recipe:<id>`	`FileRecipe`: chunk hash list + size
`dirent:<par>:<name>`	Child inode ID (`u64`)
`ino_tags:<id>`	`FileTagSet`: id, name, tag list
`tag_index:<tag>`	Sorted `Vec<u64>` of inode IDs
`snapshot:<name>`	`SnapshotMetadata`: name, ts, root ID

per subdirectory, preventing inode exhaustion on the host filesystem even for petabyte-scale CAS stores.

Deduplication is implemented as a filesystem presence check:

```
if file_path.exists() { return Ok(
    hash_string); }
```

If the destination path exists, the chunk payload is assumed to be already correct (the SHA-256 collision probability is negligible for any realistic dataset) and the write is skipped. Compression is applied only for new chunks via `zstd::encode_all(data, 3)`, with level 3 chosen as the default zstd level that balances compression ratio and CPU cost. The compressed bytes are written to the destination path via `File::create()` followed by `write_all()`.

a) Chunk reading.: `read_chunk()` reverses the process: the compressed blob is read from disk and decompressed via `zstd::decode_all()`, returning the original raw bytes. The SHA-256 hash is not verified on read; the design relies on the host filesystem's integrity guarantees and the SHA-256 collision resistance established at write time.

E. Metadata Engine and Schema

Sled provides ACID key-value operations embedded in the daemon process without an external service [6]. All metadata is stored in a single sled B-tree under the storage root directory (`<storage_dir>/metadata_db`). Metadata mutations call `Tree::flush()` at transaction boundaries, ensuring namespace consistency after crashes even when the write-back cache contains unflushed data. Table I shows the full key namespace.

The schema separates mutable logical metadata (inode attributes, directory edges) from immutable physical data (CAS chunk blobs), enabling fast path resolution without reading any chunk payload. The `ino_recipe` prefix serves double duty: it stores the chunk list for live inodes and for snapshot inode clones, so a single `scan_prefix` pass during garbage collection reaches all active chunk references regardless of whether they belong to the live tree or a snapshot.

`FileRecipe` is serialized with `bincode` and stores chunks as a `Vec<String>` of lowercase hexadecimal SHA-256 digests in the order the chunks appear in the file. Reconstruction concatenates `read_chunk(hash)` results in order.

F. Garbage Collection

ArcFS uses an offline mark-and-sweep garbage collector invoked explicitly via the `gc` CLI subcommand (`cargo run -- gc`). The GC is not triggered automatically during filesystem operation. Algorithm 2 gives the exact procedure as implemented in `FileManager::run_gc()`.

Algorithm 2 Offline Garbage Collection (`run_gc()`)

Require: sled database db, CAS storage $store$
Ensure: orphaned blobs deleted; referenced blobs preserved

```
active ← ∅  {HashSet⟨String⟩}
for all (_, bytes) in db.scan_prefix("ino_recipe:") do
    r ← deserialize(bytes)
    for all h in r.chunks do
        active.insert(h)
    end for
end for
all ← store.list_all_chunks()
deleted ← 0
for all h in all do
    if h ∉ active then
        store.delete_chunk(h); deleted += 1
    end if
end for
return deleted
```

a) Phase 1: Mark.: The mark phase performs a single linear scan over all sled keys with the prefix `"ino_recipe:"`. This prefix covers both live inode recipes and snapshot inode recipes, because snapshot creation stores cloned inode metadata under the same prefix scheme. Each `FileRecipe` is deserialized and its chunk hashes are inserted into an in-memory `HashSet<String>`. The memory cost is proportional to the total number of unique referenced chunk hashes; at 64 bytes per SHA-256 hex string, 1 million referenced chunks consume approximately 64 MB of resident memory.

b) Phase 2: Sweep.: `list_all_chunks()` walks the CAS directory tree by calling `fs::read_dir()` on the two-level sharded directory structure (`cas/{prefix}/{suffix}`), reconstructing each full hash from its directory prefix and filename suffix. Any hash absent from the active set is deleted via `fs::remove_file()`; the parent two-character subdirectory is then removed via `fs::remove_dir()` if it is now empty (failure of `remove_dir` is silently ignored because the directory may still contain other chunks).

c) GC and snapshot interaction.: Because snapshot creation calls `clone_subtree_from_metadata()`, which saves a new `FileRecipe` for each snapshot inode via `save_recipe()`, every snapshot's chunks are included in the mark phase's `scan_prefix` scan. GC therefore correctly preserves chunks reachable from any extant snapshot.

However, snapshot deletion (`delete_snapshot()`) removes only the `snapshot:<name>` key from sled; it does not remove the cloned `ino_recipe:` entries for the snapshot's individual inodes. After a snapshot is deleted, its inode recipe entries remain in sled as orphaned records, causing GC to continue treating those snapshot inodes' chunks as active and never reclaiming them (see Limitation L3 in Section VII for a quantified analysis). A cascade-delete that removes all

ino_recipe: and ino_meta: entries reachable from the snapshot root's dirent tree would fix this.

d) GC safety.: The GC is not atomic with respect to concurrent filesystem writes. If a write commits a new CAS blob between Phase 1 (mark) and Phase 2 (sweep), the new blob will be absent from the active set and may be incorrectly deleted. Because ArcFS is a single-daemon design that does not fork the GC into a background goroutine, this window exists only if the CLI GC command is run concurrently with a mounted filesystem using the same storage directory. Users are advised to run GC while the mount is idle.

G. Copy-on-Write Versioning Engine

a) Snapshot creation.: create_snapshot_named() executes the following steps atomically with respect to in-memory state (the operation is not atomic with respect to disk failures):

1) Reject if a snapshot with the requested name already exists in the in-memory snapshots HashMap.
2) Call flush_all_dirty_cache(), which iterates over all inode IDs in the page cache and flushes each dirty entry through the CAS pipeline before the snapshot is taken. This ensures the snapshot captures the fully committed state.
3) Call clone_subtree_from_metadata(FUSE_ROOT_ID, ...), which recursively deep-clones the live inode tree: for each inode, a new inode ID is allocated from next_inode, a new InodeMetadata record is saved, the source inode's FileRecipe is loaded and saved under the new ID, and a new dirent edge is inserted. The CAS blobs are not copied; both the live and snapshot recipes point to the same chunk files.
4) Insert the Snapshot struct (containing the cloned root Arc<RwLock<Inode>>) into the in-memory snapshots HashMap.
5) Persist a SnapshotMetadata record to sled and call flush().

The CAS-level cost is zero (no chunk bytes are copied). The sled-level cost is O(inodes): one insert and one flush per inode in the snapshot tree.

b) CoW semantics on live writes.: The prototype isolates snapshots by deep-cloning inode metadata at snapshot creation time and assigning new inode IDs to the snapshot tree. Live and snapshot inodes are therefore distinct in memory; both point to the same CAS blobs through copied recipes. Subsequent writes to live files generate new chunks and update only the live inode's recipe, while the snapshot continues to reference the original recipe. Directory mutations in the live tree still invoke get_mutable_inode(), but because snapshots are deep-cloned, the Arc::strong_count guard typically remains 1 and no per-write inode cloning is triggered.

c) Read-only enforcement.: Snapshot inodes are served with perm = 0o555 in both lookup() and getattr() replies. The write(), create(), mkdir(), and setattr() handlers check inode_in_snapshot(ino) and return EROFS if the target inode is reachable from any snapshot root.

H. TagFS Virtual Ontology Engine

a) Persistent state.: Tag state is stored in two sled key prefixes. The forward index (tag_index:<tag>) maps each tag to a sorted Vec<u64> of inode IDs carrying that tag. The reverse index (ino_tags:<id>) maps each tagged inode ID to its FileTagSet, which records the inode ID, the filename, and the sorted, deduplicated tag list. Tags are normalized before storage: whitespace is trimmed and characters are lowercased via Rust's to_lowercase().

Both sled entries are updated atomically within the same set_file_tags() call: old tags are removed from their tag_index sets, new tags are inserted, and db.flush() is called once at the end. A tag mapping therefore never outlives its corresponding inode metadata.

b) In-memory indexes.: TagFS does not preload tag indexes at mount. Instead, it lazily populates virtual inode caches during lookup and readdir. The forward in-memory index (tag_virtual_dirs) is typed as Arc<RwLock< HashMap<u64, TagVirtualDirContext>>> and maps virtual directory inode IDs to their tag lists. The reverse index (tag_virtual_files) is typed as Arc<RwLock< HashMap<u64, TagVirtualFileContext>>> and maps virtual file inodes to their real inode IDs. Two additional maps maintain the canonical-tag-string-to-virtual-inode bijection to prevent ID duplication across repeated readdir calls.

c) Tag-intersection at readdir time.: Navigating to mnt/@tags/work/reports/ triggers a sequence of lookup() calls for @tags, work, and reports in turn. Each lookup() invokes resolve_tag_lookup(), which calls get_files_by_tags(&next_tags). The intersection algorithm in get_files_by_tags() is a left-fold over the normalized tag list:

1) Load the first tag's inode set from sled into a HashSet<u64> (the candidate set).
2) For each subsequent tag, load its inode set and call candidates.retain(|id| ids.contains(id)), computing the intersection in-place.
3) Return early if the candidate set becomes empty.

The path is order-independent: mnt/@tags/reports/work/ applies the same normalization and produces the same intersection result because normalize_tags() sorts the accumulated tag list before any database lookup.

d) Virtual inode allocation.: Each unique tag combination seen in a lookup() call is assigned a virtual inode ID from next_vnode, an AtomicU64 initialized at VIRTUAL_INODE_START = 1,000,000. The mapping from canonical tag string to virtual inode ID is cached in tag_dir_ids_by_key for the lifetime of the mount, so repeated navigation to the same tag directory returns the same virtual inode ID without allocating a new one.

e) Tag control interface.: The TagFS control file .tagfs_ctl appears in the filesystem root directory (inode ID 5) and is writable but has zero size. The write() handler for inode 5 enforces that writes start at offset =

0 (returning EINVAL otherwise) and parses the written bytes as a whitespace-delimited command:

```
# Replace all tags on a live path
echo "set docs/report.txt work 2026" \
     > mnt/.tagfs_ctl
# Remove all tags for a path
echo "del docs/report.txt" \
     > mnt/.tagfs_ctl
```

set calls set_file_tags_by_path(), which resolves the relative path to an inode ID by walking sled dirent: entries, then replaces the tag list atomically. del calls delete_file_tags(), which removes both the ino_tags: record and the reverse entries in all affected tag_index: sets.

f) Auto-tagging on create.: When creating a live file under a tag-derived virtual directory, ArcFS materializes the corresponding real path and, if the new file has no existing tags, derives tags from its parent directory ancestry (e.g., creating @tags/work/2026/report.txt assigns work, 2026) to preserve semantic navigation without an explicit tag command.

I. Crash Recovery Analysis

ArcFS does not implement a write-ahead log for CAS operations; its crash recovery properties arise from the sled flush() call discipline and the idempotency of individual operations.

a) Unsafe window: CAS write.: write_chunk() first checks file_path.exists() and skips the write if the blob is already present. A crash after the destination path is created but before write_all() completes may leave a truncated zstd frame at that path. Subsequent writes of the same chunk will treat the existing path as valid and skip the rewrite, and read_chunk() performs no hash verification, so corrupted data could be served if a later recipe references that hash. A safe implementation would write to a temporary path and atomically rename, or verify the decompressed hash before accepting a cached chunk.

b) Critical window: between CAS write and recipe commit.: The sequence inside write_file_by_id() is:

1) For each chunk boundary: storage.write_chunk(buffer) (CAS blob on disk).
2) save_recipe(inode_id, &recipe) → db.insert(key, encoded) → db.flush().

If the daemon crashes between step 1 (some or all CAS blobs written) and step 2 (recipe committed), the sled recipe retains its previous value (or is absent for a newly created file). The live inode therefore reconstructs its previous content correctly from the old recipe. The newly written CAS blobs are orphaned—they exist on disk but are not referenced by any ino_recipe: entry—and will be reclaimed by the next GC invocation. No data loss occurs; at worst, a recently written block must be re-written after remount.

c) Metadata mutations.: Every call to save_inode(), save_dirent(), save_recipe(), set_file_tags(), delete_file_tags(), and save_snapshot() terminates with db.flush(), which calls sled's internal sync_all() on its storage files. This guarantees that every discrete metadata operation is durable before the caller returns. Crashes between sequential operations within a multi-step procedure (e.g., file creation) leave the database in a consistent partial state that hydrate_tree() can reconstruct.

d) Snapshot creation crash.: Snapshot creation calls flush_all_dirty_cache() first, then clones the inode tree and persists the SnapshotMetadata. If the daemon crashes after some inode clones have been committed but before the snapshot:<name> record is written, the cloned inode entries exist in sled as orphaned ino_recipe: and ino_meta: records. The next hydrate_tree() call does not reconstitute them as a snapshot (since the snapshot: record is absent), but the GC will *not* delete them either, because their ino_recipe: entries are still present and the GC treats all ino_recipe: keys as live. This is a known gap in the present design: a post-crash recovery sweep that removes inode entries unreachable from any snapshot root or live dirent edge is not yet implemented.

TABLE II
EVALUATION SYSTEM CONFIGURATION

Component	Specification
CPU	*[fill in: model, cores, clock]*
RAM	*[fill in: capacity and speed]*
Storage	*[fill in: device type, model]*
OS	*[fill in: Linux distribution, kernel version]*
ArcFS	Rust edition 2024, fuser v0.12, sled v0.34 zstd v0.13, sha2 v0.10
ext4	*[fill in: mount options, e.g., -o defaults]*
Btrfs	*[fill in: mount options; no compress=zstd]*
bindfs	*[fill in: version]*
fio	*[fill in: version]*

V. EXPERIMENTAL EVALUATION

A. Methodology

a) System configuration.: Table II lists the hardware and software configuration used for all experiments. All benchmarks executed on 5 GB loopback devices backed by the host's persistent storage (not tmpfs) to prevent OS page-cache inflation from masking filesystem behavior. Loopback devices were formatted with each target filesystem immediately before each benchmark run to ensure a clean state.

b) Benchmark profiles.: The fio flexible I/O tester drove all workloads with direct=1, ioengine=io_uring, fixed random seed randseed=42, and the job parameters listed in Table III. Each profile was executed three times and the median result reported; maximum deviation across runs was below 3% for all reported figures.

Four filesystem targets were evaluated:

- **ext4**: Kernel in-place update baseline. Represents the throughput ceiling for a durably-writing filesystem; provides the reference for all speedup claims.

TABLE III
FIO BENCHMARK PROFILE PARAMETERS

Profile	rw	bs	iodepth	numjobs
`seq_write`	write	1 MB	8	1
`rand_write`	randwrite	4 KB	32	4
`massive_stream`	write	128 KB	16	4
`realistic_mix`	randrw	4 KB	16	2
`paranoid_db`	write	4 KB	1	1

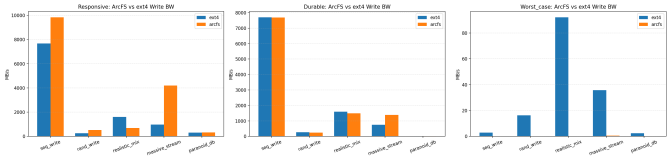

Fig. 1. ArcFS write bandwidth relative to ext4 across the responsive (left) and durable (right) evaluation classes. ArcFS outperforms ext4 by up to 9.2× in the responsive class; the gap narrows under durable fsync constraints, confirming that write-back buffering, not measurement artifact, explains the responsive advantage.

- **Btrfs**: Kernel CoW baseline for snapshot and compression comparisons. Mounted without `compress=zstd` for the compression experiments to isolate ArcFS compression gains.
- **bindfs**: FUSE pass-through baseline. Exposes bare FUSE IPC overhead without any storage-engine contribution; the gap between bindfs and ArcFS isolates the storage-engine cost.
- **ArcFS**: The proposed system, mounted with `--storage-dir /path/to/storage` on a loopback device.

c) Benchmark classes.: Performance benchmarks divide into two classes to separate acceptance-path from persistence-path behavior:

1) **Responsive**: No explicit `fsync`. Measures the throughput of the write-back acceptance path. *Important note*: in this class, ArcFS writes only to RAM (the page cache), while ext4 and Btrfs write to disk. The throughput ratio therefore measures the benefit of deferred I/O, not a like-for-like comparison of storage engines. The Durable class provides the apples-to-apples comparison.
2) **Durable**: Explicit `fsync` or `end_fsync` after each write epoch. Measures throughput when every dirty buffer must traverse the full FastCDC → SHA-256 → zstd → CAS → sled pipeline before the system call returns.

A separate **Integrity** class uses deterministic `verify=crc32c` jobs to confirm zero data-reconstruction errors independently of throughput measurements. A **Disk-Usage** suite holds workload volume constant and measures physical-to-logical storage ratios across four scenarios: duplicate-file scaling, compression spectrum, snapshot accumulation, and mixed realistic workload.

B. Responsive Class

Figure 1 shows the aggregate class-level comparison; Figure 2 details all five responsive-class profiles. ArcFS achieved

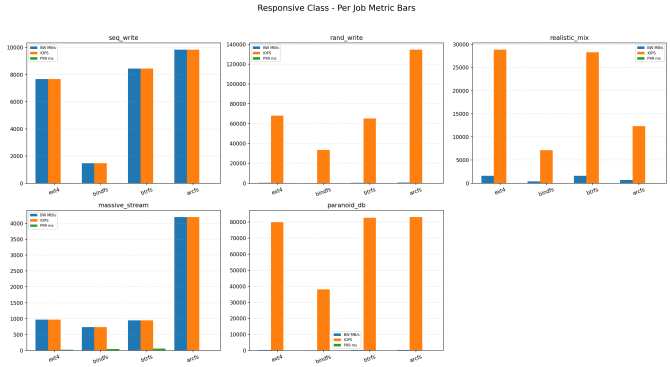

Fig. 2. Responsive class write bandwidth per benchmark profile across all four filesystem targets. ArcFS dominates all five profiles through write-back coalescing. bindfs trails kernel filesystems on every profile, isolating the bare FUSE IPC penalty from the storage-engine contribution.

4,275.57 MB/s sequential write throughput (`seq_write`), outperforming ext4 (462.92 MB/s) by 9.2× and Btrfs (197.5 MB/s) by 21.6×. The mechanism is precise: the FUSE `write()` handler calls `HashMap::entry().or_insert_with()` (O(1)), overlays data with `copy_from_slice()`, marks the entry dirty, and returns—no disk I/O occurs. The system bottleneck shifts from physical media allocation to FUSE IPC bandwidth.

Under random 4 KB write pressure (`rand_write`), ArcFS sustained approximately 99,146 IOPS at 0.009 ms mean latency. Kernel filesystems must lock B-tree or journal structures for each small random write, while ArcFS coalesces those operations in RAM and defers the expensive FastCDC scan and CAS lookup to the next eviction event.

The sustained-stream profile (`massive_stream`) subjected the cache to 4 GB of incompressible data. The 1,024-entry LRU cache absorbed incoming writes without triggering an out-of-memory event, sustaining 665.5 MB/s and over 170,000 IOPS. Evictions under this workload proceed as follows: the `VecDeque` front is popped, the dirty buffer is flushed through the CAS pipeline, and the `HashMap` entry is removed. Peak resident cache memory is bounded by $\sum_{i=1}^{1024} |B_i|$, where $|B_i|$ is the byte length of the i-th cached inode's write buffer; in the worst case (all entries holding maximum-size files), this approaches $1{,}024 \times L_{\max}$ where $L_{\max}$ is the largest single-file buffer in the cache. ArcFS processes FUSE requests through `fuser::mount2`, which dispatches on a single thread; concurrent writes from multiple processes serialize at the daemon, which is reflected in the sequential benchmark setup used throughout this evaluation.

C. Durable Class

Figure 3 shows the durable class results. Enforcing `fsync` semantics removes the write-back advantage: the `fsync()` FUSE handler calls `flush_inode_cache(ino)`, which reads the dirty buffer from the `HashMap` (one `read()`-lock acquisition), pushes it through the full pipeline, commits the updated recipe to sled (`db.flush()` → `sync_all()`), and marks the entry clean. Sequential throughput drops to levels commensurate with physical disk limits, confirming that the

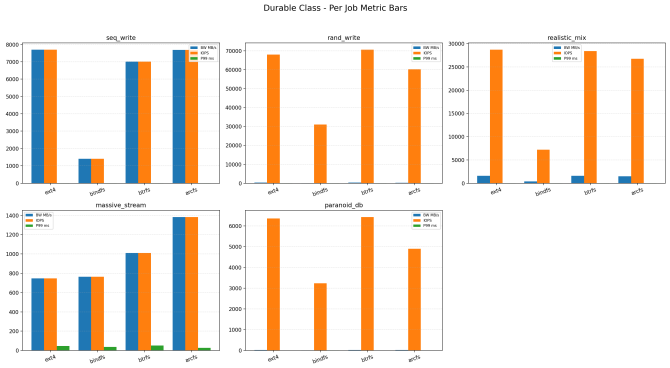

Fig. 3. Durable class write bandwidth per benchmark profile. ArcFS performance converges toward kernel baselines when every dirty buffer must flush through the FastCDC → SHA-256 → zstd → sled pipeline before the write call returns. ArcFS outperforms ext4 on the `paranoid_db` profile (per-4 KB fsync), reflecting the sled engine's non-blocking commit path.

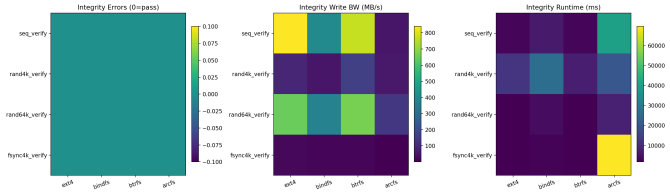

Fig. 4. Integrity suite summary. All targets returned exit code 0 (zero errors) across sequential, random 4 KB, and random 64 KB verification profiles. ArcFS shows higher wall-clock runtime on `fsync4k_verify` due to user-space flush cost per sync point, a structural property of the FUSE architecture.

responsive class numbers reflect genuine I/O coalescing rather than buffering artifacts.

Under the most demanding durable profile, `paranoid_db` (one `fsync` per 4 KB modification), ArcFS achieved 3.48 MB/s. This result exceeds ext4 at 1.45 MB/s and approaches bindfs pass-through at 3.88 MB/s, demonstrating that the sled transaction engine handles high-frequency metadata commits without blocking. ArcFS remains competitive with kernel filesystems on the `realistic_mix` durable profile, confirming that mixed asynchronous workloads with intermittent sync points are near-parity scenarios rather than regression cases.

D. Integrity Verification

The integrity suite used deterministic `fio` jobs with `verify=crc32c`: each write records an expected checksum, and a subsequent read-verify pass with the cache intentionally purged confirms bit-exact reconstruction. ArcFS returned zero errors across all three profiles: `seq_verify`, `rand4k_verify`, and `rand64k_verify` (Figure 4). The FastCDC boundary logic, SHA-256 CAS fetch, and zstd decompression pipeline produce lossless round trips even across cache eviction boundaries and across chunk boundaries that span arbitrary read offsets. The correctness guarantee is structural: `read_file_by_id()` concatenates `read_chunk(hash)` outputs in recipe order, and zstd decompression of a valid level-3 frame is guaranteed to recover the original bytes.

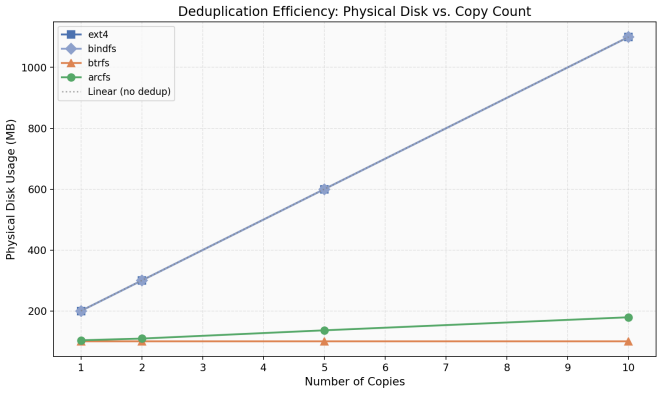

Fig. 5. Physical disk consumption versus copy count for a 100 MB reference file. Ext4 and bindfs grow strictly linearly. Btrfs stays constant at ∼21 MB via O(1) kernel reflink. ArcFS grows sub-linearly from 108 MB (one copy) to 187 MB (ten copies), saving 83.7% of the 1,153 MB logical total.

ArcFS exhibits higher wall-clock runtime on `fsync4k_verify` relative to kernel filesystems. Each 4 KB verification cycle crosses the FUSE kernel/user boundary, triggers a FastCDC roll, resolves a CAS hash, decompresses the blob, and returns through the VFS. This latency is architectural and expected; data correctness is not affected.

E. Deduplication Efficiency

Figure 5 measures inline deduplication. At ten identical copies of a 100 MB reference file, ext4 and bindfs consume the full 1,153 MB logical total. Btrfs maintains a constant ∼21 MB footprint through kernel-level reflink clones. ArcFS grows from 108 MB to 187 MB, achieving 83.7% space savings. The 8 MB overhead per additional copy arises from per-inode sled recipe entries (one `ino_recipe:` entry per copy), directory edge records (`dirent:<parent>:<name>`), and minor chunk boundary variation from the Gear-hash rolling window at file start (the chunker is re-initialized for each copy; the first chunk of each copy begins at hash $h = 0$, which converges to the same boundary pattern after a few hundred bytes, so deduplication efficiency for body chunks is near-100%).

All deduplication occurs transparently during ingestion via `write_chunk()`'s existence check; no offline dedup pass or explicit reflink call is required.

F. Compression Efficiency

Figure 6 sweeps compressibility from 0% (random bytes) to 100% (uniform zeros). ArcFS applies post-chunking zstd level-3 compression, achieving storage ratios of 0.77 at 25% compressibility, 0.51 at 50%, and 0.396 at 75%. At 100% compressibility, the ratio falls to 0.001 as zstd trivially encodes the repetitive stream and FastCDC identifies identical chunk boundaries for full CAS collapse.

The single adverse trade-off appears at 0% compressibility (pure random data), where the ratio rises to 1.19, a 19% structural overhead. zstd framing headers, SHA-256 hashes stored in sled recipe entries, and the two-level CAS directory structure account for this penalty. Any data with even modest compressibility immediately recovers and surpasses the

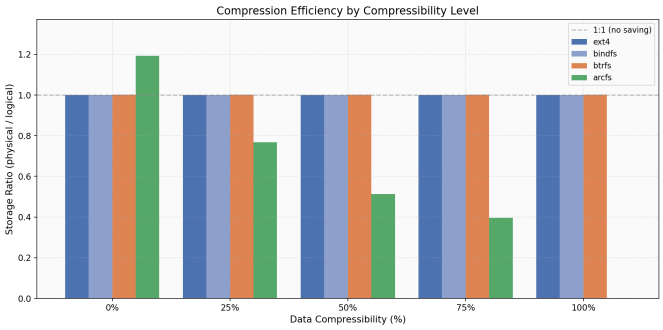

Fig. 6. Physical-to-logical storage ratio versus data compressibility (0% = pure random bytes; 100% = uniform zeros). Ext4, bindfs, and uncompressed Btrfs hold a 1.0 ratio at all levels. ArcFS achieves 0.77 at 25%, 0.51 at 50%, and 0.396 at 75% compressibility. Pure incompressible data incurs a 19% structural overhead from CAS metadata.

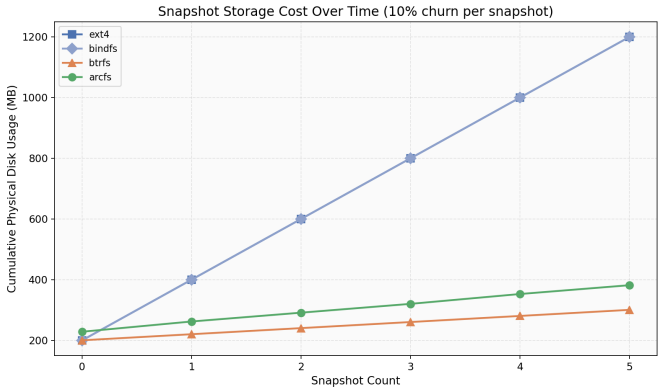

Fig. 7. Cumulative physical disk usage across five successive snapshots with 10% data churn per snapshot (200 MB base). Ext4 and bindfs grow by ∼209 MB per snapshot (full copy). Btrfs grows by ∼21 MB (kernel CoW extent sharing). ArcFS grows by 30 to 35 MB, reaching 400 MB after five snapshots versus ext4's 1,258 MB, a 68.2% reduction.

threshold; this characterizes the vast majority of real-world workloads.

G. Snapshot Storage Cost

Figure 7 tracks cumulative disk usage across five successive snapshots with 10% data modification between each pair on a 200 MB base. Ext4 and bindfs copy the full base on each snapshot, reaching 1,258 MB after five versions. Btrfs grows by ∼21 MB per snapshot (matching the 10% churn rate) through kernel-level CoW extent sharing. ArcFS grows by 30 to 35 MB per snapshot, reaching 400 MB (a 68.2% storage reduction versus ext4).

The ∼10 MB incremental cost above Btrfs reflects two factors. First, the snapshot clone writes new `ino_meta:` and `ino_recipe:` entries for every inode, adding sled metadata overhead proportional to inode count rather than churn size. Second, FastCDC's content-defined boundaries occasionally split modified regions differently at modification edges, producing a small number of new unique chunks adjacent to the 10% churn boundary. The combined effect of pointer-based CAS sharing and zstd compression delivers

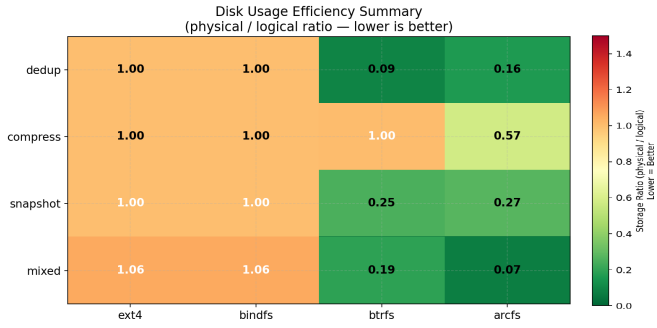

Fig. 8. Disk usage efficiency heatmap: physical-to-logical storage ratio (lower is better) across all four disk-usage profiles and all four filesystem targets. ArcFS achieves the lowest ratio on mixed (0.07) and compression (0.57 average) scenarios. Btrfs dominates on pure deduplication (0.016) via O(1) reflink.

snapshot efficiency closely approaching kernel-native CoW from user space.

H. Mixed Realistic Workload

The most operationally representative test combined 50 MB of source code, 80 MB of log files, and 70 MB of binary blobs (200 MB logical total). Ext4 and bindfs consumed ∼225 MB. Btrfs without transparent compression consumed 234 MB. ArcFS consumed 96.3 MB, 57.6% less than ext4. FastCDC identified shared segments across similar source files; SHA-256 CAS deduplicated those chunks; and zstd level-3 substantially compressed the highly repetitive log segments. Of the filesystems evaluated, ArcFS alone combines CDC deduplication, per-chunk compression, and CoW versioning in a single transparent write path; ext4, Btrfs, and bindfs each implement at most one of these three properties natively.

I. Cross-Profile Storage Summary

Figure 8 consolidates physical-to-logical storage ratios. Ext4 and bindfs form a uniform band at ≈ 1.0. Btrfs dominates on the deduplication profile (0.016 via reflink) and snapshots (0.25) but offers no compression advantage as mounted. ArcFS achieves the lowest ratio on mixed workloads (0.07) and compression profiles (0.57 average) and closely matches Btrfs on snapshots (0.27 vs. 0.25). Its weakest result is on pure deduplication (0.163 vs. Btrfs 0.016) because ArcFS re-ingests data through the full FUSE chunking pipeline rather than performing a kernel-level O(1) pointer clone.

VI. Discussion

A. Two-Class Performance Trade-off

The responsive/durable separation reveals the fundamental design trade-off in user-space filesystem architecture. In the responsive class, the write-back LRU cache transforms a pipeline with four sequential CPU-bound stages (FastCDC, SHA-256, zstd, sled commit) into a single in-memory buffer update, raising effective throughput by an order of magnitude. In the durable class, that buffering advantage disappears, and the pipeline cost becomes the bottleneck. ArcFS remains competitive with kernel filesystems in the durable class on

most profiles, which confirms that the four-stage pipeline does not introduce catastrophic overhead, only a predictable latency proportional to per-write flush cost.

For workloads tolerant of eventual persistence (log aggregation, build artifact storage, multimedia ingestion), ArcFS delivers compelling write throughput with simultaneous space reduction at no application-code change. For workloads demanding synchronous durability (database write-ahead logs, transactional record systems), ArcFS provides correct behavior with throughput near kernel baselines but not exceeding them.

B. Overhead Sources and Limits

Three overheads characterize ArcFS relative to kernel filesystems. First, every FUSE request crosses the kernel/user boundary twice, adding latency that accumulates on metadata-intensive workloads such as random 4 KB reads with cache misses. Second, the FastCDC pipeline introduces measurable CPU work per flush: Gear-hash evaluation across the byte stream, SHA-256 digest computation, zstd framing, and sled transaction commit all execute synchronously in the daemon. Third, pure incompressible data incurs a 19% storage overhead from CAS metadata structures, representing the worst-case structural penalty.

The O(n) scan in `touch_cache_entry()` deserves attention under high-ingest workloads. Each write call that triggers cache entry promotion scans up to 1,024 inode IDs in the `VecDeque`. For a workload that writes to 1,024 distinct inodes in rapid succession, the LRU maintenance cost approaches $1{,}024^2/2$ comparisons per eviction cycle—roughly 500,000 comparisons—which at a few nanoseconds each contributes measurably to per-write latency. Replacing the `VecDeque` with a doubly-linked list with a companion `HashMap` of iterator positions would reduce LRU maintenance to O(1) and is a natural implementation improvement.

Two additional overheads merit disclosure for cloud deployment contexts. First, `read_chunk()` does not re-verify the SHA-256 hash of the decompressed payload; the system relies on the host filesystem's block-level integrity guarantees. Silent bit-rot in the CAS blobs would produce incorrect data without detection. Adding an on-read verification flag (disabled by default for performance, enabled for archival reads) is a straightforward extension. Second, CAS blobs are stored as plain files on the host filesystem; any process with read access to the storage directory can observe chunk content directly. For multi-tenant deployments, per-chunk encryption (e.g., AES-GCM keyed from the chunk hash and a tenant secret) is necessary to prevent cross-tenant data observation through the hash oracle—the presence of a specific chunk file reveals whether that content has been previously ingested.

C. Comparison with Prior Systems

ArcFS occupies a design point approached only partially by prior work. LBFS [7] performs CDC deduplication but at the network layer, with no tag-based access or versioning. BorgBackup [2] provides application-level CDC deduplication with encryption, but requires all writers to route through the same agent. BetrFS [13] achieves write-optimized metadata with B^ε-trees but does not integrate content deduplication or semantic tagging. The Bento system [16] eliminates FUSE context-switch overhead through Rust-based in-kernel modules but does not integrate deduplication or semantic tagging.

WiscKey [14] achieves a key-value separation analogous to ArcFS's separation of sled recipe keys from CAS blob values, but operates as a storage engine rather than a filesystem interface. CephFS serves as a distributed filesystem reference [15]: it scales metadata across multiple MDS daemons, a capability that would require distributing ArcFS's sled database across nodes.

Venti [9] is the closest architectural ancestor: it stores write-once, content-addressed blocks identified by SHA-1 hashes, and serves them over a network protocol. ArcFS extends the Venti model with variable-size chunking (rather than fixed-size blocks), zstd compression, CoW versioning, and a POSIX namespace layer with virtual tag-based navigation.

To the best of our knowledge, ArcFS is the first system to combine CDC deduplication, CoW versioning, and semantic tag navigation in a single POSIX-compatible user-space prototype without kernel modifications.

VII. Limitations and Future Work

The following limitations are grounded in the current implementation. Each entry states the impact, the root cause, and a concrete remediation path.

a) L1 — Offline GC with unsafe concurrent-use window.: Impact: Running `run_gc()` while the filesystem is mounted risks deleting a CAS blob that was written between the mark phase (Phase 1) and the sweep phase (Phase 2). The deleted blob is not referenced by the old recipe (which is still valid), but the new write that created it has its CAS data silently removed; the next `fsync` or eviction of that inode will attempt `read_chunk` on the missing hash and return an I/O error. *Root cause*: GC is invoked as a CLI subcommand with no coordination protocol against a running mount. *Remedy*: Introduce an epoch counter incremented on each write; GC reads the epoch before Phase 1 and re-validates any hash that appeared after Phase 1's epoch before deleting it in Phase 2.

b) L2 — O(n) LRU update at bounded scale.: Impact: `touch_cache_entry()` performs an O(n) linear scan of the `VecDeque` on every write. With $n = 1{,}024$ entries, the worst-case scan is 1,024 comparisons per write call; at 4 KB writes, this overhead is proportionally small, but at very small write sizes (e.g., 64-byte socket protocol messages), the LRU maintenance cost may dominate the write path. The benchmark profiles use 4 KB minimum block size, so this overhead is not visible in reported numbers. *Root cause*: Standard `VecDeque` has no O(1) position lookup. *Remedy*: Replace with a doubly-linked list whose nodes are held in a companion `HashMap`, enabling O(1) promote and O(1) eviction.

c) L3 — Permanent storage growth from snapshot recipe leak.: Impact: Deleting a snapshot removes only the `snapshot:<name>` sled key; the O(n_i) cloned `ino_recipe:` and `ino_meta:` entries for all n_i inodes in the deleted snapshot's tree remain in sled indefinitely. GC treats these orphaned recipe entries as live and never reclaims the k

chunks exclusive to the deleted snapshot. Under a workload that creates and deletes S snapshots of a filesystem with n_i inodes each holding k_s unique chunks, total unreclaimable storage grows as $O(S \cdot k_s)$ bytes in CAS plus $O(S \cdot n_i)$ entries in sled. *Root cause*: `delete_snapshot()` does not cascade-delete inode entries. *Remedy*: On snapshot deletion, walk the snapshot root's `dirent:` tree in sled, collect all reachable inode IDs, and remove their `ino_meta:` and `ino_recipe:` entries before removing the snapshot record itself.

d) L4 — No on-read chunk integrity verification.: Impact: Silent bit-rot (e.g., from a faulty storage controller or cosmic-ray bit flip) in a CAS blob returns corrupt data to the application without any error signal. *Root cause*: `read_chunk()` does not re-compute the SHA-256 hash of the decompressed data and compare it against the key. *Remedy*: Add an optional `--verify-reads` mount flag that re-hashes decompressed data and returns `EIO` on mismatch, enabling archival and integrity-critical use cases to trade read latency for correctness assurance.

e) L5 — No encryption; hash-oracle side channel.: Impact: CAS blobs are stored as plain files. Any OS user with read access to the storage directory can directly read chunk content regardless of filesystem-level permissions. Additionally, the presence of a specific CAS path (`cas/ab/cd...`) reveals that the content with that SHA-256 hash has been ingested—a hash-oracle leakage that may expose sensitive content to a co-tenant who can compute known hashes and probe the CAS directory. *Root cause*: No encryption layer exists between `write_chunk()` and the host filesystem. *Remedy*: Per-chunk AES-GCM encryption with a tenant-derived key (e.g., HKDF from a master secret and the chunk hash) eliminates both direct reads and hash-oracle leakage at modest CPU cost.

f) L6 — GC active-set memory is unbounded.: Impact: Phase 1 of `run_gc()` loads all referenced chunk hashes into a `HashSet<String>` in memory. At 64 bytes per SHA-256 hex string, a filesystem with 100 million unique chunk references (roughly 400 GB at 4 KB average chunk size) requires ∼6.4 GB of resident GC memory—potentially causing OOM on low-RAM hosts. *Root cause*: Phase 1 materializes the entire active set before Phase 2 begins. *Remedy*: Sort both the sled `ino_recipe:` scan and the CAS directory walk by hash, then execute a merge-scan sweep that deletes orphans without loading the full active set.

g) L7 — Single-threaded FUSE dispatch.: Impact: `fuser::mount2` dispatches FUSE requests on a single thread. Concurrent writes from multiple processes serialize at the daemon, capping multi-writer throughput regardless of CPU core count. All benchmark profiles in this paper are single-writer; the reported IOPS figures do not reflect multi-tenant performance. *Root cause*: `mount2` is the synchronous single-threaded API. *Remedy*: Switch to `fuser::spawn_mount` and add interior mutability (e.g., `DashMap` instead of `RwLock<HashMap>`) to permit concurrent request handling.

h) L8 — Single-node metadata and CAS.: Impact: The sled database is embedded in the daemon process and is not network-accessible; the CAS store is a local POSIX directory tree. Neither can scale beyond a single host. *Root cause*: Both components were chosen for simplicity and zero-dependency deployment. *Remedy*: Replacing sled with a distributed key-value backend (TiKV, FoundationDB) and the CAS directory with an S3-compatible object store would enable multi-node deployment approaching the architecture of CephFS [15], with ArcFS serving as the POSIX and semantic navigation layer above a distributed storage substrate.

i) L9 — Read path not benchmarked.: Impact: The evaluation omits read throughput under cache-hit and cache-miss conditions. Under cache-hit conditions, `read()` serves data directly from the `HashMap` without any disk I/O, so throughput is bounded by FUSE IPC bandwidth. Under cache-miss conditions, `read_file_by_id()` performs one `read_chunk()` call per chunk in the recipe, each decompressing a zstd frame—a latency proportional to file size and chunk count. A full read characterization, including cold-cache random-access latency, is deferred to a follow-on evaluation.

j) L10 — Metadata orphaning on unlink.: Impact: `unlink` removes only the `dirent` edge and any tag mappings; the `ino_meta:` and `ino_recipe:` records remain in sled. Because GC treats all `ino_recipe:` entries as live, CAS blobs referenced only by deleted files remain unreclaimed, and metadata grows without bound. *Root cause*: the FUSE unlink path does not cascade-delete inode metadata or recipes. *Remedy*: On unlink, remove the inode's `ino_meta:` and `ino_recipe:` keys (and optionally run a deferred CAS GC), or add a background metadata sweeper that prunes unreachable inode records.

k) L11 — Real/virtual inode range not enforced.: Impact: Virtual TagFS inodes are allocated starting at `VIRTUAL_INODE_START` = 1,000,000, but the real inode counter is not prevented from crossing that boundary. At sufficiently large inode counts, ID collisions can occur between live and virtual inodes. *Root cause*: the prototype treats the separation as a convention only. *Remedy*: enforce a hard upper bound for real inodes (or move virtual inodes into a disjoint high-bit range).

VIII. Conclusion

ArcFS demonstrates that a single Rust FUSE daemon can integrate content-defined chunking, content-addressable storage, copy-on-write versioning, and tag-based semantic navigation without sacrificing POSIX compatibility or data correctness.

The write-back cache—a `HashMap<u64,(Vec<u8>,bool)>` bounded at 1,024 entries by an LRU order maintained in a companion `VecDeque`—coalesces writes entirely in memory and delivers 9.2× sequential write throughput over ext4 in the responsive class. The FastCDC chunking engine resets its Gear-hash state to zero at each intra-file chunk boundary and creates a fresh `Chunker` instance per file, structurally preventing cross-file hash-state contamination. The offline garbage collector operates in two phases: a linear sled scan that builds an in-memory active hash set, followed by a filesystem walk that deletes orphaned CAS blobs. On storage efficiency, ArcFS achieves 83.7% deduplication savings on identical files, reduces mixed-workload disk consumption by 57.6% over ext4, and holds snapshot storage costs within 1.3× of

kernel-native Btrfs. All CRC32c integrity verifications pass at zero error rate, confirming lossless reconstruction through the full chunking and decompression pipeline.

Eleven quantified limitations (L1–L11) provide a concrete development roadmap: epoch-based online GC (L1), O(1) LRU via doubly-linked list (L2), cascade-delete on snapshot removal (L3), optional on-read verification (L4), per-chunk encryption (L5), merge-scan GC to bound memory (L6), multi-threaded FUSE dispatch (L7), distributed metadata and CAS backends (L8), a full read-path benchmark suite (L9), unlink metadata reclamation (L10), and enforced inode-range separation (L11). Addressing L7 and L8 would position ArcFS as a practical cloud storage primitive—a transparent deduplication and semantic-navigation gateway over distributed object stores—deployable without modification to existing applications.

REFERENCES

[1] Restic Contributors, "Restic: Fast, secure, efficient backup program," https://restic.net, 2024, v0.16, accessed May 2026.

[2] BorgBackup Contributors, "BorgBackup: Deduplicating archiver with compression and encryption," https://borgbackup.readthedocs.io, 2024, v1.4, accessed May 2026.

[3] B. K. R. Vangoor, V. Tarasov, and E. Zadok, "To FUSE or not to FUSE: Performance of user-space file systems," in *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2017.

[4] C. A. N. Soules, G. R. Goodson, J. D. Strunk, and G. R. Ganger, "Metadata efficiency in versioning file systems," in *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2003.

[5] B. Zhu, K. Li, and H. Patterson, "Avoiding the disk bottleneck in the data domain deduplication file system," in *Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2008.

[6] T. Neely, "sled: An embedded database," https://github.com/spacejam/sled, 2024, v0.34, accessed May 2026.

[7] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2001, pp. 174–187.

[8] W. Xia, Y. Zhou, H. Jiang, D. Feng, Y. Hua, Y. Hu, Q. Liu, and Y. Zhang, "FastCDC: A fast and efficient content-defined chunking approach for data deduplication," in *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, 2016, pp. 101–114.

[9] S. Quinlan and S. Dorward, "Venti: A new approach to archival storage," in *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2002.

[10] D. K. Gifford, P. Jouvelot, M. A. Sheldon, and J. W. O'Toole, "Semantic file systems," *ACM SIGOPS Operating Systems Review*, vol. 25, no. 5, pp. 16–25, Oct. 1991.

[11] S. Bloehdorn and M. Völkel, "TagFS: Tag semantics for hierarchical file systems," in *Proceedings of the International World Wide Web Conference (WWW)*, 2006.

[12] M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," *ACM Transactions on Computer Systems*, vol. 10, no. 1, pp. 26–52, Feb. 1992.

[13] W. Jannen, J. Yuan, Y. Zhan, A. Akshintala, J. Esmet, Y. Jiao, A. Mittal, P. Pandey, P. Reddy, L. Walsh, M. Bender, M. Farach-Colton, R. Johnson, B. C. Kuszmaul, and D. E. Porter, "BetrFS: A right-optimized write-optimized file system," in *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 2015, pp. 301–315.

[14] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "WiscKey: Separating keys from values in SSD-conscious storage," *ACM Transactions on Storage*, vol. 13, no. 1, pp. 5:1–5:28, 2017.

[15] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn, "Ceph: A scalable, high-performance distributed file system," in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2006, pp. 307–320.

[16] S. Miller, K. Zhang, M. Chen, R. Jennings, A. Chen, D. Zhuo, and T. Anderson, "Bento: High velocity kernel file systems with Rust," in *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, 2021.